NATIONAL SPACE HACKATHON 2026 - Orbital Debris Avoidance & Constellation Management System

Yashasvi Gupta, Aastha Shah, T Vishal, Sai Parcha

April 3, 2026

1 Introduction

This project was an ambitious undertaking by our team, aimed at tackling the complex problem of satellite orbit maneuvering, debris avoidance, and efficiency. We invested significant effort in designing the architecture, exploring various approaches, and understanding the underlying dynamics, which deepened our appreciation for the challenges involved in space systems. Throughout this journey, we not only enhanced our technical knowledge but also developed a strong curiosity and passion for the domain. While the problem itself is vast and extends beyond the scope of a limited timeline, the system presented here reflects what we were able to build within the deadline. Regardless of the outcome, the experience was highly rewarding—we learned a great deal, enjoyed the process, and consider this a meaningful success in itself.

2 Contents -

- **General Architecture - Layer 1 - Data Ingestion - Layer 2 - Orbit prediction and risk of collision - Layer 3 - Maneuvering - Bonus Innovations - Visualization and User Interface - Numerical Methods and Spatial Optimization algorithms - Conclusion - References**

3 General Architecture

The system is designed as a **modular, distributed pipeline** for real-time space object monitoring and analysis, consisting of three primary layers: a high-performance ingestion layer, an intelligent processing layer, and a visualization interface. At the core of the architecture lies a centralized data layer that ensures seamless communication and decoupling between components.

The ingestion layer is implemented using Go (go-adapter), which exposes a REST API (`POST /api/telemetry`) to receive high-frequency telemetry data from satellites and debris tracking systems. Leveraging Go's lightweight concurrency model, the service processes incoming data using a worker pool architecture, enabling parallel handling of large volumes of streaming data with minimal latency. The processed telemetry—comprising spatial coordinates, velocity vectors, object type, and timestamps—is then stored in a MongoDB Atlas database. This database acts as a **shared data backbone (or "glue layer")**, allowing independent system components to read and write data efficiently without tight coupling.

The backend processing layer, implemented in Python, is responsible for domain-specific logic, including orbital computations, state management, and higher-level analysis. Modules within this layer (such as physics simulations and state storage) operate on the ingested data to generate insights like trajectory estimation and potential risk indicators. Finally, a front-end interface presents these outputs through an interactive dashboard, enabling users to visualize spatial data and system insights effectively. This layered and decoupled architecture ensures scalability, flexibility, and ease of integration, making the system well-suited for real-time, data-intensive space monitoring applications.

4 Layer 1 - Data Ingestion

4.0.1 1] COMPLETE FLOW

Step 1: Telemetry Ingestion API receives data. This is the **entry point** of data. Source of data/ telemetry comes from:

- satellites
- debris tracking systems

Our system receives data like:

```
{
"_id": "DEB-99421",
"r": { "x": 4500.2, "y": -2100.5, "z": 4800.1 },
"v": { "x": -1.25, "y": 6.84, "z": 3.12 },
"type": "DEBRIS",
"updated_at": "timestamp"
}
```

Endpoint:

`POST /api/telemetry`

4.0.2 Step 2: Go parses the request and sends it to the worker pool

- Validates incoming data
- Converts into internal structure

Instead of processing immediately, it is pushed into a **worker queue.**

4.0.3 Step 3: The worker processes it

Each worker:

- normalizes values (coordinates)
- validates ranges

4.0.4 Step 4: Store in MongoDB

- Insert/update document
- Acts as persistent storage

Other layers (Python AI and U.I.) will read the data from Mongo.

4.0.5 2] INTERNAL GO ARCHITECTURE

Core components

`API Handler → Channel Queue → Worker Pool → Mongo Writer`

How it actually works -

4.0.6 1. Channel (Queue)

```
var telemetryQueue = make(chan Telemetry, 1000)
```

Acts like a buffer between incoming data and processing.

4.0.7 2. Worker Pool (32 workers)

```
for i := 0; i < 32; i++ { go worker(telemetryQueue) }
```

4.0.8 3. Worker Function

```
func worker(queue chan Telemetry) { for data := range queue { processed := preprocess(data) saveTo
```

4.0.9 4. API Handler

```
func ReceiveTelemetry(c *gin.Context) { var data Telemetry c.BindJSON(&data) telemetryQueue <- dat
```

The main reason for this design is that it **decouples ingestion from processing.** Without this:

API waits → slow → crashes under load But we need the API to be fast and the processing to happen asynchronously.

3] WHY GO

The system contains many objects (satellites + debris) whose positions, velocities, and other parameters are continuously updated. This is why Go is chosen as the preferred language for this layer. Main advantages include:

4.0.10 1. Goroutines

- Goroutines are not the same as Threads. Threads are heavy; they take up more memory. Whereas Goroutines are lightweight ($\tilde{2}$ KB). Thus, we can run thousands simultaneously, resulting in Parallel Throughput.

4.0.11 2. Scheduler

- Scheduler allows us to distribute work across CPU cores and balance the load during the Go runtime automatically. This enables efficient and fast processing and storage of incoming data.

4.0.12 3. Non-blocking I/O

- While one worker waits for the DB write, others keep working. This allows for an ideal CPU + I/O Balance.

4.0.13 4. Burst Handling

- Telemetry is NOT steady and exhibits sudden spikes (during transitions from no coverage to coverage areas). Workers absorb bursts more efficiently.

4] WHY MONGODB

MongoDB acts like a central control room where all subsystems deposit and retrieve data independently.

4.0.14 1. Flexible Schema

- The objects, as we have defined in our architecture, include satellites and debris, which keep evolving, and their values keep changing and updating. Mongo handles this naturally and easily.

4.0.15 2. JSON-native

- Our data is already in a JSON-like format. Mongo stores data in a similar format, so no transformation is required.

4.0.16 3. Fast Writes

- Telemetry data incoming has a high write rate, and Mongo is optimized for insert-heavy workloads.

4.0.17 4. Spatial Data Friendly

- We are storing (x, y, z) coordinates and velocity vectors. Mongo has built-in features for geographic coordinate queries and indexing, which can be useful for the system in the future.

5] LAYER SUMMARY

The ingestion layer is implemented using Go to handle high-throughput telemetry streams via a REST API. Incoming data is asynchronously processed using a 32-worker goroutine pool, ensuring efficient parallel handling of a large amount of continuous data. The processed telemetry is stored in MongoDB Atlas, which acts as a centralized, schema-flexible data layer enabling seamless integration with downstream AI and visualization components.

5 Layer 2 - Orbit Prediction and Risk of Collision

5.0.1 1] COMPLETE FLOW

Step 1: Data is fetched from MongoDB. This acts as the **input layer** for all computations. The data includes:

- satellite telemetry
- debris telemetry
- velocity and position vectors

Each object is represented as:

```
{
"_id": "SAT-001",
"r": { "x": 4200.5, "y": -1800.2, "z": 5100.7 },
"v": { "x": -2.1, "y": 6.2, "z": 2.9 },
"type": "SATELLITE",
"updated_at": "timestamp"
}
```

5.0.2 Step 2: Threat Pre-Clustering

Instead of comparing every satellite with every debris object, nearby debris objects are grouped into clusters.

- Reduces comparisons from $O(N^2)$
- Converts object-level comparison into cluster-level filtering
- Implemented using spatial indexing (KD-Tree)

5.0.3 Step 3: Relative Motion Modeling

For each satellite-cluster pair, motion is transformed into a relative reference frame (RTN).

- $\vec{r}_{rel} = \vec{r}_{deb} - \vec{r}_{sat}$
- simplifies trajectory comparison
- reduces computational complexity

5.0.4 Step 4: Trajectory Propagation

Future positions are predicted over a time window using RK4 + J2 perturbation:

- physics-based propagation
- adaptive timestep near collision zones
- supports short-term (real-time) and long-term prediction

5.0.5 Step 5: Closest Approach Computation

The system computes:

- Minimum distance d_{min}
- Time of Closest Approach (TCA)

Using:

$$d(t) = |\vec{r}_{sat}(t) - \vec{r}_{deb}(t)|$$

5.0.6 Step 6: Collision Probability Estimation

Beyond distance, a probabilistic collision risk is computed:

$$P_c = f(d_{min}, v_{rel}, \sigma)$$

Where:

- d_{min} = closest distance
- v_{rel} = relative velocity
- σ = positional uncertainty

This aligns with real-world conjunction analysis systems.

5.0.7 Step 7: Risk Scoring

Each event is assigned a composite risk score:

$$Risk = w_1 \cdot \frac{1}{d_{min}} + w_2 \cdot v_{rel} + w_3 \cdot P_c + w_4 \cdot priority$$

- incorporates physics + operational importance
- enables prioritization of threats

5.0.8 Step 8: Alert Generation

If thresholds are exceeded:

- CRITICAL: $d_{min} < 0.1$ km
- WARNING: $d_{min} < 5$ km

Example:

```
{
"satellite_id": "SAT-1",
"threat_id": "DEB-45",
"min_distance": 120,
"Pc": 0.012,
"risk_score": 0.87,
"priority": "HIGH"
}
```

5.0.9 Step 9: Store Results

Outputs stored:

- collision alerts (CDM)
- trajectory predictions
- risk scores and probabilities

—

5.0.10 2] INTERNAL ARCHITECTURE

```
Data Fetcher → Clustering Engine → Propagation Engine →
Conjunction Analyzer → Risk Engine → Alert Generator
```

5.0.11 Key Enhancement

- KDTree-based screening ($O(N \log N)$)
- TCA refinement using numerical optimization

—

5.0.12 3] WHY THIS DESIGN

- **Scalability**: clustering + KDTree reduces complexity
- **Predictive Intelligence**: forecasts future risks
- **Accuracy**: physics-based propagation
- **Decision-Ready Output**: provides Pc + risk scores

—

5.0.13 4] TECH STACK

- Python (FastAPI backend)
- NumPy / SciPy
- KDTree (spatial indexing)

—

5.0.14 5] LAYER SUMMARY

This layer transforms raw telemetry into actionable collision intelligence using clustering, physics-based propagation, and probabilistic risk modeling.

6 Layer 3 - Maneuvering

6.0.1 1] COMPLETE FLOW

Step 1: Collision alerts from Layer 2 are consumed.

Step 2: Threat prioritization based on:

- risk score
- collision probability
- time-to-collision

Step 3: Maneuver strategy selection:

- Hohmann Transfer
- Radial / Along-track burn
- Plane Change

Step 4: Delta-V computation:

$$\Delta V = |\vec{v}_{target} - \vec{v}_{current}|$$

Step 5: Burn splitting:

- $\Delta V \leq 15$ m/s per burn
- scheduled with cooldown (600 s)

Step 6: Fuel computation:

$$\Delta m = m\left(1 - e^{-\frac{\Delta V}{I_{sp} g_0}}\right)$$

Step 7: Maneuver scheduling and execution.

—

6.0.2 2] INTELLIGENT DECISION LAYER

- evaluates multiple strategies
- selects minimum fuel solution
- considers time constraints

Example:

```
Selected Maneuver: Radial Burn
Reason: Fastest separation before TCA
Fuel Cost: 5 kg
```

—

6.0.3 3] WHY THIS DESIGN

- ensures autonomous decision making
- minimizes fuel usage
- guarantees constraint satisfaction

—

6.0.4 4] LAYER SUMMARY

This layer converts collision risks into optimal maneuver strategies using physics-based calculations and intelligent decision-making.

7 Bonus Innovations

7.0.1 Key Additions

- Collision Probability (Pc)
- Adaptive timestep RK4
- KDTree spatial filtering
- Decision-aware risk scoring
- Maneuver strategy optimization
- Fuel-aware planning
- Parallel processing
- Trajectory caching
- Early termination logic
- Relative motion (RTN frame)

7.0.2 Advanced Enhancements

- Maneuver reasoning engine
- Before/after maneuver prediction
- Autonomous decision logs
- Risk trend analysis

7.0.3 Impact

- reduces complexity from $O(N^2)$ to $O(N \log N)$
- enables real-time decision making
- improves accuracy near collision events

8 Visualization and User Interface

8.0.1 Enhanced Features

- Real-time satellite and debris tracking
- Collision probability visualization (Pc)
- Risk trend graphs
- Maneuver preview (before vs after)
- Ground track visualization

8.0.2 Decision Intelligence UI

- Recommended maneuver + reasoning
- Fuel forecast and mission lifetime
- Decision window (time remaining)
- Satellite health score

8.0.3 System Monitoring

- CDM alerts panel
- maneuver timeline
- fuel usage tracking

8.0.4 Summary

The UI transforms complex backend outputs into intuitive, decision-ready insights for operators.

9 Numerical Methods and Spatial Optimization Algorithms

9.0.1 Core Methods

- RK4 integration for orbital propagation
- J2 perturbation modeling
- Adaptive timestep control

9.0.2 Collision Detection

- KDTree screening ($O(N \log N)$)
- TCA optimization
- Euclidean distance thresholding

9.0.3 Optimization Techniques

- ROI filtering
- clustering
- relative motion frame

9.0.4 Maneuver Optimization

- fuel minimization using Tsiolkovsky equation
- constraint-based burn planning

9.0.5 Final Summary

The system integrates physics, numerical methods, and spatial optimization to achieve scalable, accurate, and real-time orbital intelligence.

10 Conclusion

This project presents a scalable and modular system for real-time space object monitoring, collision detection, and risk analysis. By integrating a high-performance data ingestion layer, an intelligent processing engine, and an interactive visualization interface, the system is capable of handling large volumes of dynamic orbital data efficiently.

Throughout the design, emphasis was placed on balancing computational efficiency with predictive accuracy. Techniques such as clustering, adaptive propagation, and risk-based prioritization enable the system to focus on critical scenarios while maintaining real-time performance. The modular architecture also ensures flexibility, allowing future integration of advanced components such as reinforcement learning-based maneuver planning and AI-driven decision systems.

While the current implementation represents a strong foundation, the problem of space traffic management is inherently complex and continuously evolving. This work serves as a stepping stone toward more autonomous, intelligent, and globally coordinated systems for ensuring safe and sustainable operations in Earth's orbit.

11 References

1. Vallado, D. A., *Fundamentals of Astrodynamics and Applications*, Microcosm Press.
2. Curtis, H. D., *Orbital Mechanics for Engineering Students*, Elsevier.
3. NASA Orbital Debris Program Office, *Orbital Debris Quarterly News*.
4. ESA Space Debris Office, *Space Debris Mitigation Guidelines*.

5. MongoDB Documentation, https://www.mongodb.com/docs/

6. Golang Documentation, https://go.dev/doc/

7. Scikit-learn Documentation, https://scikit-learn.org/

8. Poliastro Library Documentation, https://docs.poliastro.space/

9. Kelso, T.S., "Analysis of the 2007 Chinese ASAT Test and the Impact of its Debris on the Space Environment," *Advanced Maui Optical and Space Surveillance Technologies Conference.*

10. Kessler, D. J., and Cour-Palais, B. G., "Collision Frequency of Artificial Satellites: The Creation of a Debris Belt," *Journal of Geophysical Research.*